

OOP and Design Patterns (CSCI 375)
Student Showcase (Final Project) Rubric

1. Project Title: File Sorter Application

Instructions:

1. There are 10 technical requirements (worth 100 points) to grade the project and the team presentation.
2. For each requirement, use 0 - 10 scale in the Score column (0-6: Fail, 7: C (Average), 8: B (Above Average), 9: A (Good), (10: Excellent)
3. Use the *Notes* section to jot down any observations that may help in grading and justification.

Team and Technical Project Requirement	Score
1. Use 4+1 Views to explain the software design to the audience. Notes: Had all diagrams, informative and easy to reference. Did not cover every single thing but sufficient information to understand and use the program. Diagrams were split evenly among all members, class diagram produced by the creator of the class.	10/10
2. Use of 3 Design Patterns -- presentation clearly stated and briefly explained design patterns use. Common design patterns are Iterator, Decorator, Observer, Strategy, Command, State, Singleton, Adapter, Façade, Flyweight, Abstract Factory, Composite, Template, MVC, etc. Notes: The GUI class was a singleton. Observer/Call backs were the basis for keeping info up-to-date and quick, which I had helped a lot on. Factory pattern was attempted to be another design, using helper functions for creating classes, files, and directories.	9/10
3. Use of Fundamental OOD Concepts- e.g., Inheritance, Abstraction, Getters and Setters, Overloading, Attributes and Methods, etc. Notes: All necessary class contained getters/setters, attributes, and methods	10/10
4. Software management – good usage of management, communication and tracking tools e.g., Gant chart, Kanban board, GitHub Project, Clickup, Discord, Slack, etc. Notes: We would meet frequently to discuss progress and delegating assignments. Group chat used to communicate and ask questions when needed. Github was used to create branches for different parts of the process. Group effort.	10/10

5. Documentation – clear, easy to follow documentation, UML diagrams are complete, and notations are correct; explanation of objects interaction is clear and complete. Notes: Documentation is direct, organized, and accessible. Everyone equally documented their work and classes.	10/10
6. Testing and CI-CD pipeline – automatically generates test data using hypothesis, usage of mocking/patching, provides code coverage and Python type check (mypy) reports, CI-CD, Docker, etc. Notes: Included checks for format, style, and type. Test coverage fully completed. Test done inside a Docker container. Hypothesis used where possible. All equally provided help, although test related to tkinter was difficult and required help.	8/10
7. Clean code and Modularity – project uses appropriate software and system design; the code follows best practices (pep8), is well organized and refactored into packages, modules, classes, etc. Notes: A modeler was intended and without it, causes a lot to go on in the GUI class. pep8/mypy/etc was done to the best of my abilities, reviewed, and corrected until no errors were found. Packages and modules used for make commands and modules in multiple files to make the easy to read.	9/10
8. Project requirements and execution -- clearly stated functional and technical requirements, project adequately challenging for sophomore-junior students; project demo was clear and concise, etc. Notes: Program ran well; the purpose of fulfilling all requirements of a project start to finish was adhered to. It was challenging enough but greatly influenced how I approach projects.	10/10
9. Team presentation -- all members participated in presentation, used the visual and oral presentation techniques and tools to engage audience, etc. Notes: We equally presented the project and tried to be really clear, without too much excess information.	10/10

10. Individual Contributions – briefly explain how and what each member contributed to the successful execution of the project. Notes: As stated throughout this document, many parts were separated equally between us all. Certain things were delegated to some, such as the GUI being predominantly complete by Alex. It was intuitive and easy to use. I did help some on it, using AI due to the lack of familiarity I have with tkinter. Same for some GUI testing and EventHandler testing. Classes were split between us, diagrams too. I did classes for Semester and assisted others. Parker did classes and tags. Alex did GUI and EventHandler. I wrote some tests for others, mainly to help update coverage to 100%. Parker also updated documentation throughout the project. I felt the work spread out appropriately and adequately.	10/10
11. BONUS: Above and beyond – Team went beyond the above list e.g., great User Interface, use of Database, security, real-world application, real-world client delight and interaction, etc. Notes: I focused a lot on the project as a whole, adding everything from the start to the end as learned from class assignments. I would say we were above and beyond but detailed enough. There was an added feature that acted as an explorer but lacked the intended search feature. It's clean and functional but was not focused on at the end. It uses a wrapper to pick between the two programs, using different but similar (almost identical) classes since they no longer matched by the end. Ultimately, I think I would've preferred to use the explorer into a portable fs creation tool that is meant to be a helper to a program requiring it. The refactoring to be more generic wasn't viable time wise. I would've used it, I have changed every semester and now there's messes everywhere. Tagging would have been extremely helpful when referring to my old programs. It was tough to find certain things.	5/10
Total Score Note: Max score can be 110 due to 10 BONUS points.	(70) (18) (8) (5) 101/100